
CS 301

High-Performance Computing

Assignment 6 & 7 — Parallel Interpolation & Parallel Interpolation with Particle Mover

Diya Patel (202301216)

Yesha Joshi (202301462)

April, 2026

0 Contents

1	Introduction and Problem Overview	3
1.1	Input Configurations	3
2	Mathematical Formulation	3
2.1	Weight Computation	3
2.2	Accumulation	4
3	Pseudocode and Illustrative Diagram	5
3.1	Pseudocode of the Parallel Implementation	5
3.2	Illustrative Diagrams	6
4	Analysis Questions from Lab 6	7
4.1	Pseudocode and Illustrative Diagram	7
4.2	Parallel Implementation and Race Condition Handling	7
4.2.1	Race Condition Analysis	7
4.2.2	Chosen Strategy: Thread-Private Accumulation with Critical Merge	8
4.2.3	Memory Cost and Trade-offs	8
4.2.4	Alternative Approaches Considered	9
4.3	Speedup and Execution Time Graphs	9
4.3.1	Performance Analysis	11
4.4	Parallel Efficiency and Scalability	12
4.4.1	Efficiency Definition	12
4.4.2	Efficiency Table	12
4.4.3	Efficiency Plot	13
4.4.4	Scalability Discussion	13
4.5	Comparison with Serial Baseline and Maximum Speedup	14
4.5.1	Maximum Speedup Table	14
4.5.2	Graphs	15
4.5.3	Observations	15
4.6	Explanation of Observed Speedup Using HPC Concepts	16
4.6.1	Detailed Explanation	16
4.6.2	Cross-Configuration Comparison Graphs	17
4.7	Further Optimisations with Unlimited HPC Resources	18
4.8	Load Imbalance and Performance Bottlenecks	19
4.9	Load Imbalance	19
4.10	Performance Bottlenecks	19
4.11	Alternative Approaches	19
5	Analysis Questions from Lab 7	20
5.1	Pseudocode and Illustrative Diagram	20
5.2	Parallel Implementation Approach and Race Condition Handling	20
5.3	Performance Graphs	21
5.4	Parallel Efficiency and Scalability	22
5.5	Comparison with Serial Version	24
5.6	Explanation of Speedup and Observations	24
5.7	Further Optimisation with Unlimited HPC Resources	24
5.8	Load Imbalance and Performance Bottlenecks	25

5.9	Alternative Approaches	25
5.10	Interpolation vs. Mover Phase Analysis	26
6	Conclusion	27

1 Introduction and Problem Overview

The assignment 6 focuses on parallelising a bilinear scatter-point-to-mesh interpolation algorithm using OpenMP. Building on the framework introduced in Assignment 03, we are given N randomly distributed particles inside a unit-square domain $[0, 1] \times [0, 1]$, each carrying a function value $f_i = 1$. The goal is to accumulate these values onto a structured $M \times M$ mesh using a four-point bilinear interpolation kernel.

The assignment 7 extends the particle-in-cell (PIC) simulation framework introduced in Assignments 03 and 04. The focus shifts from parallelising the *scatter* interpolation (particle $\rightarrow$ mesh) to also parallelising the *gather* reverse interpolation (mesh $\rightarrow$ particle), while retaining the previously parallelised scatter and mover steps. The complete pipeline — interpolation, mover, and reverse interpolation — must be fully parallelised using OpenMP.

The key challenge in parallelising this kernel is that multiple particles can map to the same grid cell or adjacent cells, leading to concurrent updates on shared grid points. Therefore, naive parallel updates may produce **race conditions**.

The assignment also encourages experimentation with data structures, memory layout, and OpenMP parallelisation strategies.

1.1 Input Configurations

Table 1: Five benchmark configurations prescribed in the assignment.

Config	N_x	N_y	Particles	Iterations
C1	250	100	0.9 million	10
C2	250	100	5 million	10
C3	500	200	3.6 million	10
C4	500	200	20 million	10
C5	1000	400	14 million	10

2 Mathematical Formulation

Given a set of scattered points $\mathcal{S} = \{(x_i, y_i, f_i) \mid i = 1, \dots, N\}$ within a 1×1 domain, and a structured mesh

$$\mathcal{G} = \{(X_i, Y_j) \mid X_i = i \Delta x, Y_j = j \Delta y, i, j = 0, \dots, M-1\}, \quad \Delta x = \frac{X_{\max}}{M}, \quad \Delta y = \frac{Y_{\max}}{M},$$

the bilinear interpolation distributes the value of each scattered point to the four nearest grid nodes.

2.1 Weight Computation

For a particle located at (x_i, y_i) within the grid cell bounded by the nodes (X_i, Y_j) , (X_{i+1}, Y_j) , (X_i, Y_{j+1}) , and (X_{i+1}, Y_{j+1}) , the local distances from the lower-left corner

(X_i, Y_j) are defined as:

$$l_x = x_i - X_i, \quad l_y = y_i - Y_j. \quad (1)$$

Using bilinear interpolation, the contribution of the particle is distributed among the four surrounding grid nodes with the following weights:

$$w_{i,j} = (\Delta x - l_x)(\Delta y - l_y), \quad (2)$$

$$w_{i+1,j} = l_x(\Delta y - l_y), \quad (3)$$

$$w_{i,j+1} = (\Delta x - l_x) l_y, \quad (4)$$

$$w_{i+1,j+1} = l_x l_y. \quad (5)$$

These weights represent the fractional contribution of the particle to each grid node, proportional to the relative area opposite to the corresponding node within the cell.

2.2 Accumulation

The mesh values are updated as:

$$F(X_i, Y_j) += w_{i,j} \cdot f_i, \quad F(X_{i+1}, Y_j) += w_{i+1,j} \cdot f_i, \quad \dots$$

Since $f_i = 1$ for all particles, each weight is accumulated directly.

Note that the cell indices are computed on-the-fly:

$$i = \left\lfloor \frac{x_i}{\Delta x} \right\rfloor, \quad j = \left\lfloor \frac{y_i}{\Delta y} \right\rfloor$$

3 Pseudocode and Illustrative Diagram

3.1 Pseudocode of the Parallel Implementation

Algorithm 1 Parallel Interpolation + Mover Pipeline

Require: Points $P[0..N - 1]$, Mesh $F[0..GX \cdot GY - 1]$, iterations T

```

1: Initialise  $P$  from input
2: for  $t = 1$  to  $T$  do
  — Interpolation (Scatter) —
3:    $F[:] \leftarrow 0$ 
4:   #pragma omp parallel
5:   Allocate thread-local grid  $F_{\text{local}}[:] \leftarrow 0$ 
6:   #pragma omp for
7:   for each particle  $p$  do
8:     if  $P[p]$  is void then continue
9:     end if
10:     $(i_x, i_y) \leftarrow \lfloor x_p / \Delta x \rfloor, \lfloor y_p / \Delta y \rfloor$ 
11:    Clamp  $(i_x, i_y)$  within bounds
12:     $l_x \leftarrow x_p - i_x \Delta x; l_y \leftarrow y_p - i_y \Delta y$ 
13:    Compute weights  $w_{00}, w_{10}, w_{01}, w_{11}$ 
14:    Accumulate weights into  $F_{\text{local}}$ 
15:  end for
16:  Merge all  $F_{\text{local}}$  into global  $F$  (critical section)
  — Normalisation —
17:  Compute  $F_{\min}, F_{\max}$  (parallel reduction)
18:  if  $F_{\max} \neq F_{\min}$  then
19:    #pragma omp parallel for
20:     $F \leftarrow 2(F - F_{\min}) / (F_{\max} - F_{\min}) - 1$ 
21:  end if
  — Mover (Gather) —
22:  #pragma omp parallel for
23:  for each particle  $p$  do
24:    if  $P[p]$  is void then continue
25:    end if
26:     $(i_x, i_y) \leftarrow \lfloor x_p / \Delta x \rfloor, \lfloor y_p / \Delta y \rfloor$ 
27:    Clamp  $(i_x, i_y)$  within bounds
28:    Compute  $l_x, l_y$  and weights  $w_{00}, w_{10}, w_{01}, w_{11}$ 
29:    Interpolate  $F_p$  from neighbouring grid values
30:    Update position:  $x_p += F_p \cdot \Delta x, y_p += F_p \cdot \Delta y$ 
31:    if  $x_p \notin [0, 1]$  or  $y_p \notin [0, 1]$  then
32:      mark  $P[p]$  as void
33:    end if
34:  end for
  — Denormalisation —
35:  if  $F_{\max} \neq F_{\min}$  then
36:    #pragma omp parallel for
37:     $F \leftarrow (F + 1) / 2 \cdot (F_{\max} - F_{\min}) + F_{\min}$ 
38:  end if
39: end for
40: Write  $F$  to output

```

3.2 Illustrative Diagrams

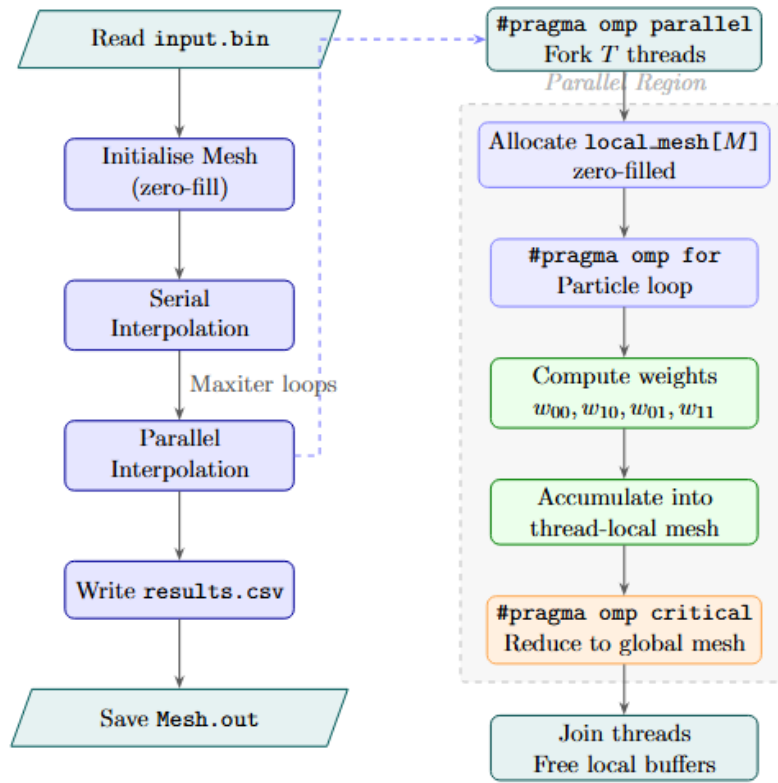


Figure 1: Parallel interpolation workflow using OpenMP privatisation.

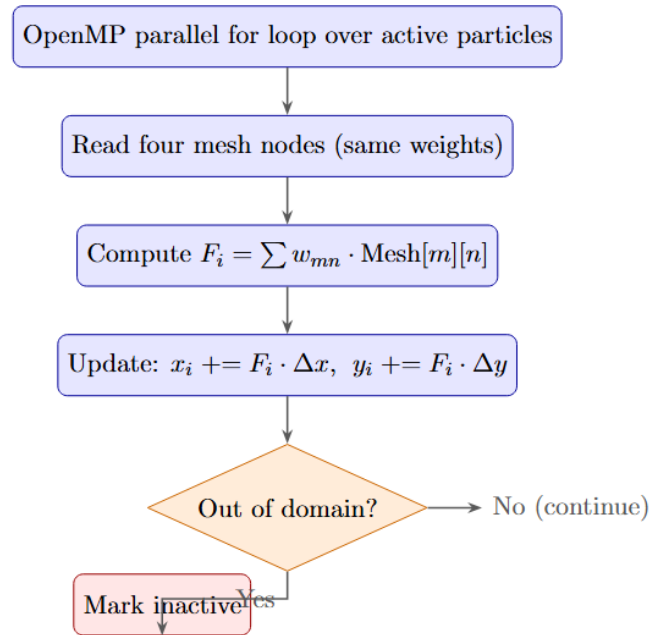


Figure 2: Gather / mover phase: mesh values are interpolated back to each active particle and positions are updated.

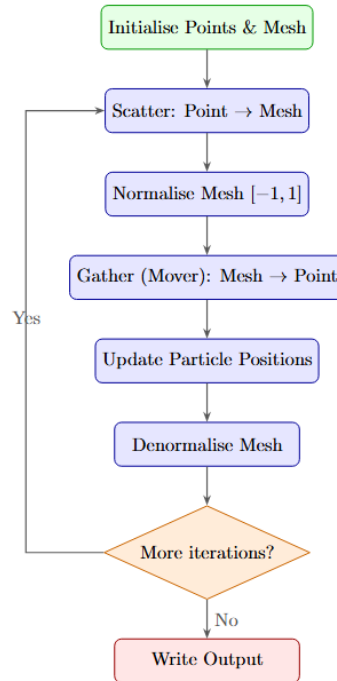


Figure 3: High-level simulation pipeline executed for each iteration.

4 Analysis Questions from Lab 6

4.1 Pseudocode and Illustrative Diagram

Pseudocode is given in **Section 3.1** and flow diagrams in **Sections 3.2**.

Key Advantages of this Strategy:

- Eliminates race conditions during particle updates.
- Provides good scalability for large particle counts.
- Maintains deterministic and numerically correct results.
- Simpler to implement than fine-grained locking or atomic updates.

Limitation: The final reduction step introduces synchronization overhead, which becomes more visible at higher thread counts.

4.2 Parallel Implementation and Race Condition Handling

4.2.1 Race Condition Analysis

In the serial implementation, each particle contributes bilinear weights to four neighboring grid nodes. When the particle loop is parallelized using `#pragma omp for`, multiple threads may attempt to update the same grid location simultaneously.

This occurs because particles located in the same cell or adjacent cells may share one or more common grid nodes. As a result, concurrent updates to the global mesh produce a **read-modify-write race condition**.

Thread A reads $F[k]$, Thread B reads $F[k]$, both update and write $\Rightarrow$ one update may be lost.

Such races lead to non-deterministic and incorrect interpolation results, making explicit synchronization or data privatization necessary.

4.2.2 Chosen Strategy: Thread-Private Accumulation with Critical Merge

To eliminate shared-write conflicts, a **thread-private mesh** strategy was used.

Each OpenMP thread allocates its own `local_mesh` array of size `GRID_X × GRID_Y`, initialized to zero. During the particle loop, threads write only to their private mesh, so no synchronization is required in the main interpolation phase.

After all particles have been processed, each thread merges its local mesh into the global mesh inside a `#pragma omp critical` section.

```

1 #pragma omp parallel
2 {
3     double *local_mesh =
4         (double *)calloc(GRID_X * GRID_Y, sizeof(double));
5
6     #pragma omp for
7     for (int p = 0; p < NUM_Points; p++) {
8
9         /* compute i, j, lx, ly, weights */
10
11         int base = j * GRID_X + i;
12
13         local_mesh[base] += w00;
14         local_mesh[base + 1] += w10;
15         local_mesh[base + GRID_X] += w01;
16         local_mesh[base + GRID_X + 1] += w11;
17     }
18
19     #pragma omp critical
20     {
21         for (int i = 0; i < GRID_X * GRID_Y; i++)
22             mesh_value[i] += local_mesh[i];
23     }
24
25     free(local_mesh);
26 }

```

Listing 1: Core parallel region from `utils.cpp`

This method guarantees correctness because only one thread performs the final merge at a time, while the expensive particle loop remains fully parallel.

4.2.3 Memory Cost and Trade-offs

The privatization strategy requires one additional mesh per thread.

Hence, extra memory usage is:

$$T \times \text{GRID_X} \times \text{GRID_Y} \times 8 \text{ bytes}$$

where T is the number of threads.

For configuration C5: $1001 \times 401 \approx 4.01 \times 10^5$ nodes

For 16 threads: $16 \times 1001 \times 401 \times 8 \approx 51$ MB which is acceptable on modern multicore systems.

However, the final reduction inside the critical section requires:

$$O(\text{GRID_X} \times \text{GRID_Y})$$

work per thread, and therefore becomes a scalability bottleneck at higher thread counts.

4.2.4 Alternative Approaches Considered

Table 2: Comparison of race-condition handling strategies.

Strategy	Description	Trade-off
Privatisation (chosen)	Each thread updates a private mesh followed by final reduction	Low contention; extra memory cost
Atomic updates	Use <code>#pragma omp atomic</code> for each grid update	Simple implementation; heavy contention for dense workloads
Domain decomposition	Assign mesh regions to threads and process only associated particles	No reduction overhead; requires particle reordering
Colouring checkerboard	/ Update independent cell groups in separate phases	Avoids races; introduces sequential multi-pass execution

Among these methods, privatization provided the best balance between simplicity, correctness, and performance for the given assignment constraints.

4.3 Speedup and Execution Time Graphs

Figures 4 present the core performance graphs for all five configurations across 2, 4, 8, and 16 threads on the Lab PC and HPC Cluster.

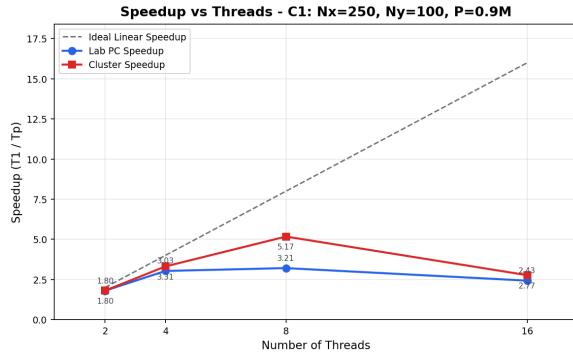
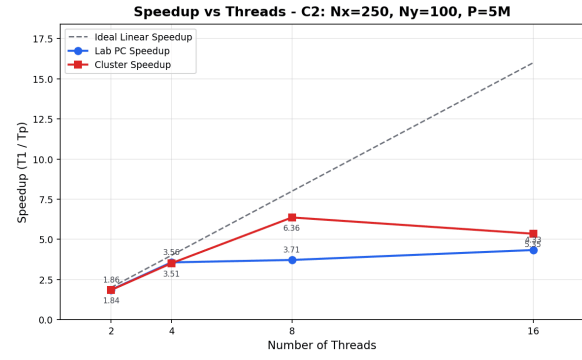
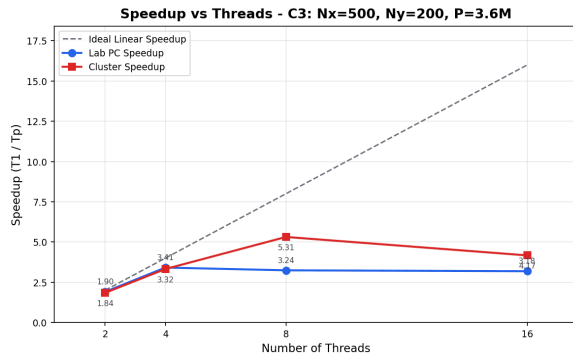
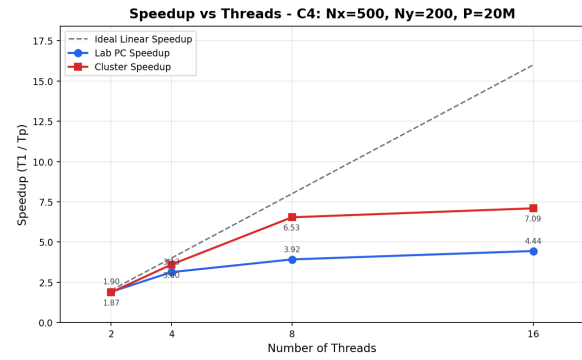
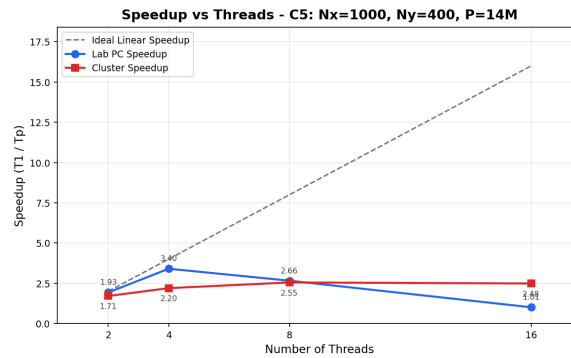
(a) C1: $N_x = 250, N_y = 100, P = 0.9M$ (b) C2: $N_x = 250, N_y = 100, P = 5M$ (c) C3: $N_x = 500, N_y = 200, P = 3.6M$ (d) C4: $N_x = 500, N_y = 200, P = 20M$ (e) C5: $N_x = 1000, N_y = 400, P = 14M$

Figure 4: Speedup vs. number of threads for all five configurations. Dashed grey line denotes ideal (linear) scaling. Blue curves correspond to Lab PC results, while red curves correspond to HPC cluster results.

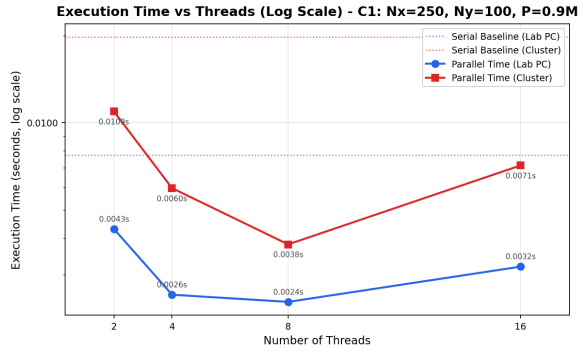
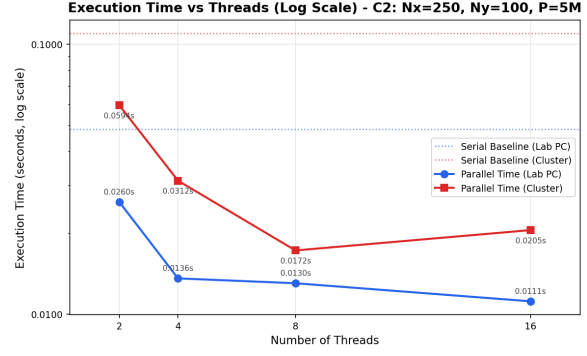
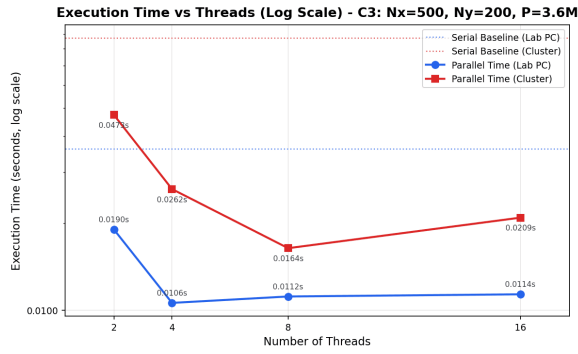
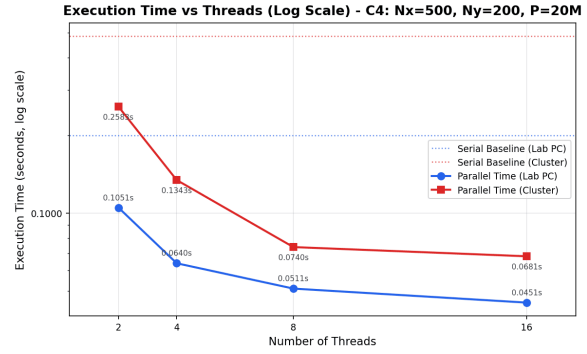
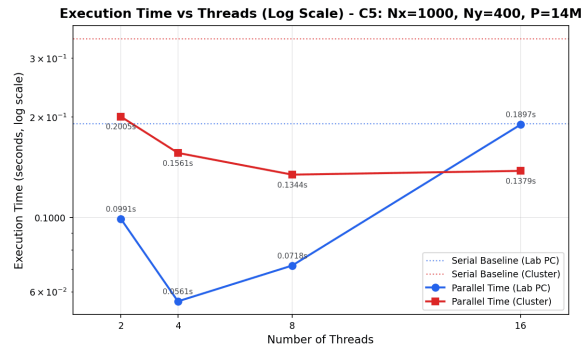
(a) C1: $N_x = 250, N_y = 100, P = 0.9M$ (b) C2: $N_x = 250, N_y = 100, P = 5M$ (c) C3: $N_x = 500, N_y = 200, P = 3.6M$ (d) C4: $N_x = 500, N_y = 200, P = 20M$ (e) C5: $N_x = 1000, N_y = 400, P = 14M$

Figure 5: Execution time vs. number of threads for all five configurations. Horizontal dashed lines indicate the serial baseline for each environment.

4.3.1 Performance Analysis

From Figure 4, speedup increases initially as the thread count rises from 2 to 4 or 8, confirming that the interpolation kernel benefits from parallel execution. However, speedup remains below the ideal linear trend in all cases due to OpenMP overhead, thread creation cost, memory traffic, and the final reduction stage.

For smaller workloads such as C1, performance saturates quickly because the computation per thread becomes too small compared to parallel overhead. Hence, increasing threads beyond 8 leads to reduced speedup.

For moderate and large workloads (C2–C4), better scalability is observed since more

particles are available for parallel processing. Among all cases, C4 achieves the highest cluster speedup of 7.09 at 16 threads, showing that larger workloads better amortize thread overhead.

Configuration C5 exhibits poor scaling despite large particle count because the mesh size is much larger (1000×400), causing higher memory usage, cache pressure, and expensive reduction of thread-private meshes.

From Figure 5, execution time decreases initially with increasing thread count, then increases after the optimal point in several cases. This indicates that synchronization and reduction overhead dominate beyond 8 threads.

Overall, the HPC cluster consistently outperforms the Lab PC due to stronger processors, better memory bandwidth, and more efficient parallel execution. The most practical thread count for this assignment is 8 threads, where strong speedup is obtained without excessive overhead.

4.4 Parallel Efficiency and Scalability

4.4.1 Efficiency Definition

Parallel efficiency measures how effectively the available threads are utilised and is defined as:

$$E(p) = \frac{S(p)}{p} \times 100\%$$

where $S(p)$ is the measured speedup using p threads.

4.4.2 Efficiency Table

Table 3: Parallel efficiency (%) for Lab PC and HPC Cluster across all thread counts.

Config	2 Threads		4 Threads		8 Threads		16 Threads	
	PC	Cluster	PC	Cluster	PC	Cluster	PC	Cluster
C1	89.9%	90.2%	75.7%	82.8%	40.1%	64.7%	15.2%	17.3%
C2	92.8%	92.1%	89.1%	87.7%	46.4%	79.5%	27.1%	33.4%
C3	95.2%	92.1%	85.3%	83.0%	40.5%	66.4%	19.9%	26.0%
C4	95.2%	93.5%	78.2%	89.9%	49.0%	81.7%	27.7%	44.3%
C5	96.3%	85.5%	85.0%	54.9%	33.2%	31.9%	6.3%	15.5%

4.4.3 Efficiency Plot

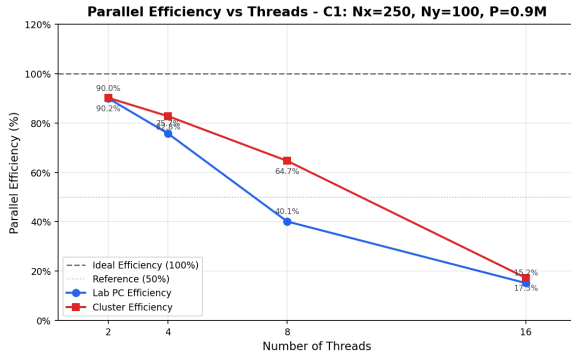
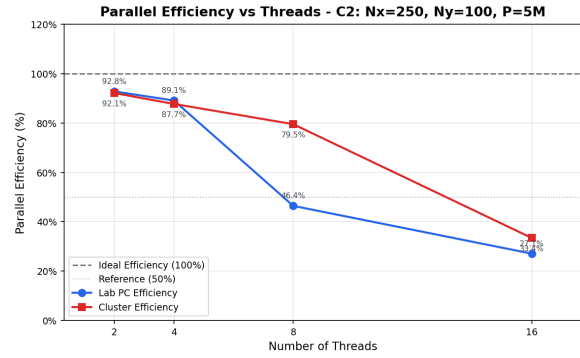
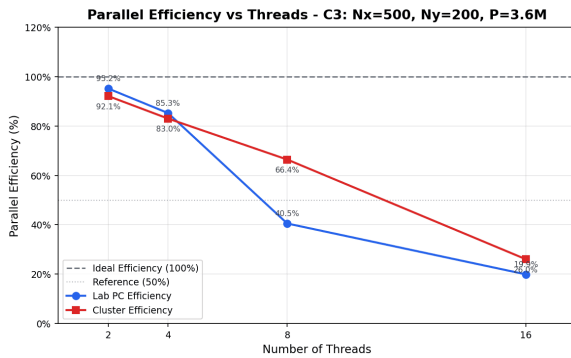
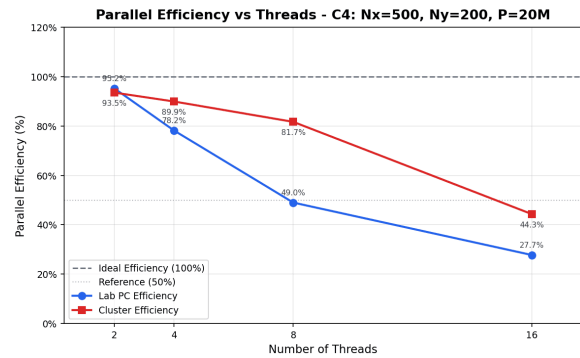
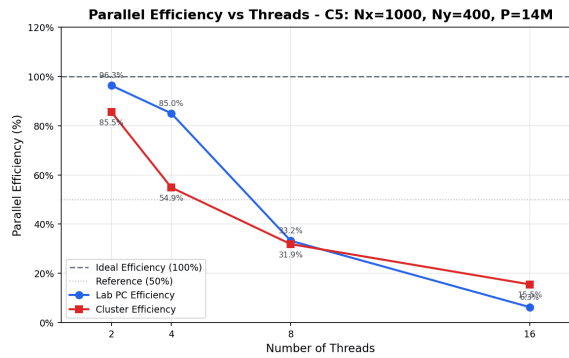
(a) C1: $N_x = 250$, $N_y = 100$, $P = 0.9M$ (b) C2: $N_x = 250$, $N_y = 100$, $P = 5M$ (c) C3: $N_x = 500$, $N_y = 200$, $P = 3.6M$ (d) C4: $N_x = 500$, $N_y = 200$, $P = 20M$ (e) C5: $N_x = 1000$, $N_y = 400$, $P = 14M$

Figure 6: Parallel efficiency for all configurations. Efficiency decreases as thread count increases due to synchronization cost, reduction overhead, cache effects, and memory bandwidth limitations. The HPC cluster sustains higher efficiency than the Lab PC for most larger workloads.

4.4.4 Scalability Discussion

- **2–4 Threads:** Most configurations maintain high efficiency (approximately 75–96%), indicating that the parallel workload is large enough to offset thread-management overhead at lower thread counts.
- **8 Threads:** Efficiency starts to decline on the Lab PC (typically 33–49%), while the

HPC cluster still sustains strong efficiency (64–82%) for larger workloads such as C2 and C4. This demonstrates better multicore scalability on the cluster system.

- **16 Threads:** Efficiency drops significantly in many cases. For smaller workloads such as C1, the overhead of thread creation, scheduling, and reduction becomes comparable to useful computation. For C5, the larger mesh size causes the final reduction of thread-private meshes to become memory-bandwidth limited.
- **Effect of Problem Size:** Larger particle-count cases (C2 and C4) scale better than smaller workloads because the fixed OpenMP overhead is amortized over more computational work.
- **Overall Scalability:** The implementation exhibits good strong scaling up to 8 threads and moderate scaling up to 16 threads for sufficiently large workloads. Beyond this point, synchronisation overhead and memory contention limit further performance gains.
- **Best Observed Case:** The highest practical performance is achieved for C4 on the HPC cluster, where both speedup and efficiency remain comparatively strong even at high thread counts.

Hence, the thread-private mesh strategy successfully removes race conditions and provides strong scalability for moderate thread counts, while the final reduction stage becomes the primary bottleneck at higher concurrency.

4.5 Comparison with Serial Baseline and Maximum Speedup

4.5.1 Maximum Speedup Table

Table 4: Best parallel speedup observed per configuration compared with serial execution.

Config	Environment	Serial Time (s)	Best Parallel (s)	Speedup	Threads
C1	Lab PC	0.007749	0.002416	3.21×	8
C1	Cluster	0.019715	0.003812	5.17×	8
C2	Lab PC	0.048309	0.011147	4.33×	16
C2	Cluster	0.109462	0.017205	6.36×	8
C3	Lab PC	0.036197	0.010614	3.41×	4
C3	Cluster	0.087124	0.016398	5.31×	8
C4	Lab PC	0.200242	0.045103	4.44×	16
C4	Cluster	0.483308	0.068141	7.09×	16
C5	Lab PC	0.190927	0.056136	3.40×	4
C5	Cluster	0.342734	0.134405	2.55×	8

To evaluate the benefit of parallelisation, the execution time of the OpenMP implementation was compared against the corresponding serial baseline for each configuration. The speedup is computed as:

$$S(p) = \frac{T_1}{T_p}$$

where T_1 is the serial execution time and T_p is the parallel execution time using p threads.

4.5.2 Graphs

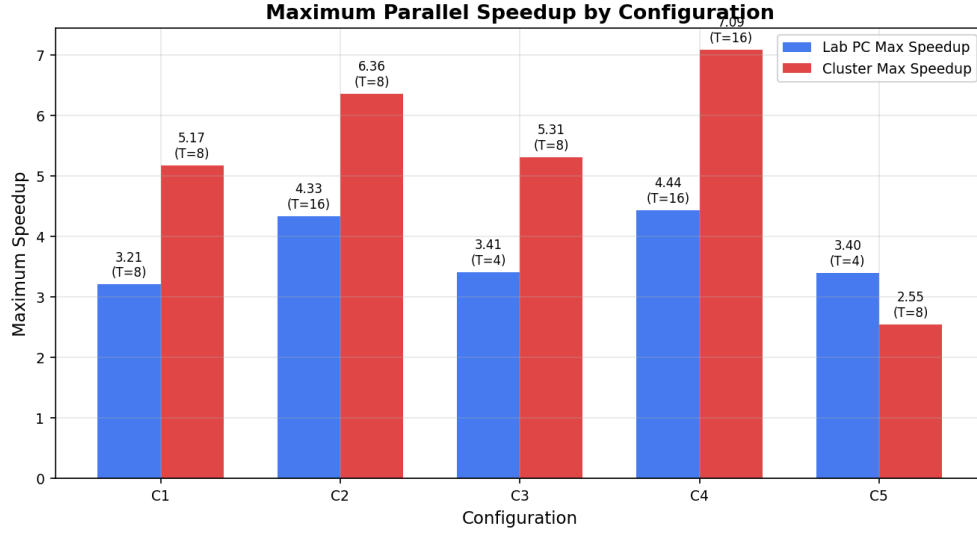


Figure 7: Maximum speedup achieved for each configuration. The thread count at which the best speedup occurred is annotated above each bar.

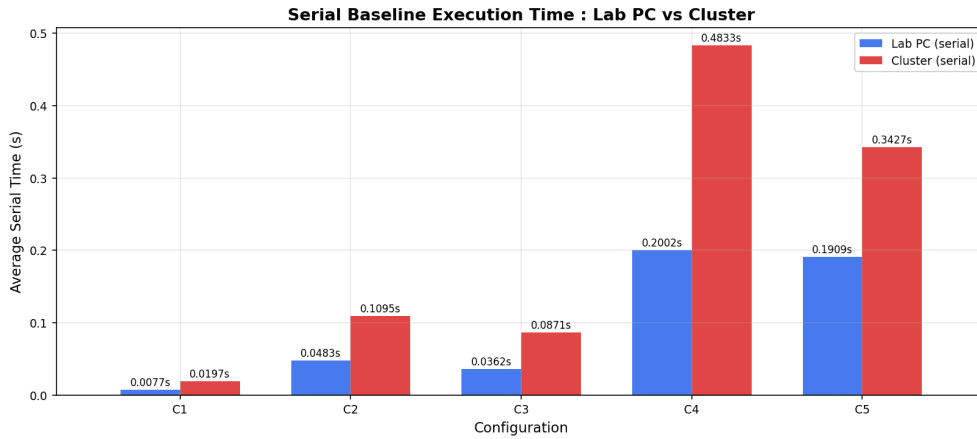


Figure 8: Serial baseline execution time comparison between Lab PC and HPC Cluster. Larger workloads naturally require longer runtime.

4.5.3 Observations

- The parallel version outperforms the serial version in all tested cases.
- The highest overall speedup is obtained for **C4 on the HPC Cluster**, reaching **7.09×** using 16 threads.

- Medium-to-large workloads (C2, C3, C4) achieve better speedup than very small workloads (C1), because thread overhead becomes less significant when more particles are processed.
- The HPC Cluster gives higher maximum speedup than the Lab PC for C1–C4, showing superior multicore utilisation.
- Configuration C5 behaves differently: the Lab PC achieves $3.40\times$, while the cluster reaches only $2.55\times$. This indicates that very large mesh reductions become memory intensive.

Hence, the OpenMP parallel implementation consistently improves performance over the serial code, with the best gains observed for sufficiently large workloads.

4.6 Explanation of Observed Speedup Using HPC Concepts

The measured speedups can be explained using core High Performance Computing principles such as parallel overhead, memory hierarchy, synchronisation cost, Amdahl's law, and hardware architecture.

4.6.1 Detailed Explanation

1. Independent Particle Computation

Each particle interpolation is mathematically independent. After using thread-private meshes, threads process particles simultaneously without race conditions. This creates strong data parallelism and good scaling at low thread counts.

2. Low Thread Counts Give Best Efficiency

At 2 and 4 threads, computation dominates overhead. Thread creation, scheduling, and reduction costs are relatively small, so speedup is close to ideal.

3. Amdahl's Law

A portion of the code remains serial, especially the final merge of local meshes:

$$S_{\max} = \frac{1}{f + \frac{1-f}{p}}$$

where f is the serial fraction. As thread count increases, this serial part limits further acceleration.

4. Reduction Bottleneck

Each thread maintains a private mesh and later merges it into the global mesh inside a critical section. Only one thread can perform this merge at a time, so contention increases with more threads.

5. Memory Bandwidth Limitation

Large grids such as C5 require copying and summing many mesh values. Multiple threads compete for memory bandwidth, reducing the benefit of extra cores.

6. Cache Effects

For moderate grid sizes (C2–C4), local meshes fit better in cache, reducing memory latency and improving performance. Very large grids exceed cache capacity, causing more cache misses.

7. Hyper-threading / Logical Core Saturation

The Lab PC likely uses logical cores beyond physical cores at high thread counts. Once physical cores are fully occupied, extra threads share resources and yield smaller gains.

8. Cluster Hardware Advantage

The HPC cluster likely provides faster memory channels, larger cache, cleaner OS environment, and stronger CPU cores. Therefore, it sustains higher speedup for most workloads.

4.6.2 Cross-Configuration Comparison Graphs

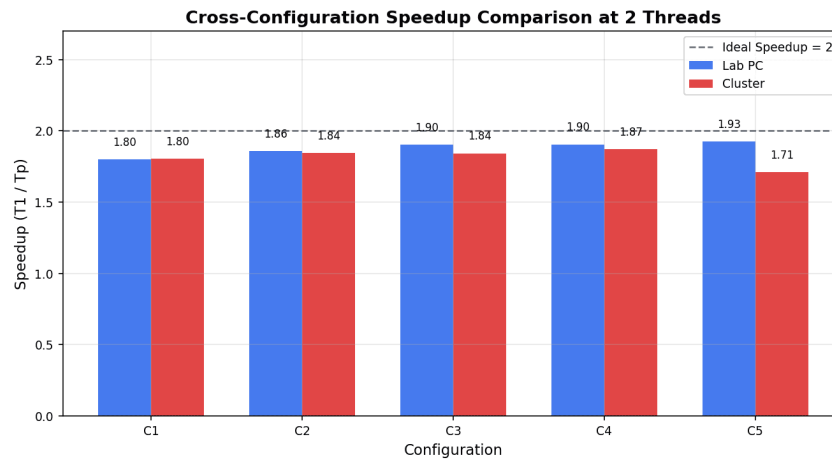


Figure 9: Speedup comparison across configurations at 2 threads. Performance is close to ideal for all cases.

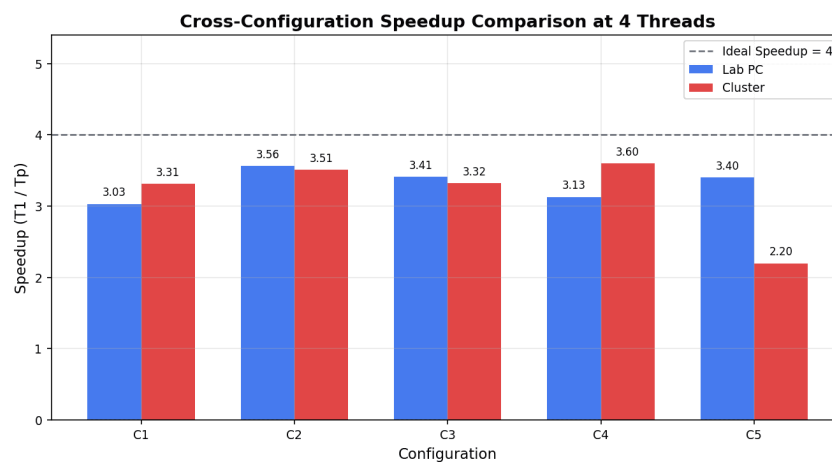


Figure 10: Speedup comparison across configurations at 4 threads. Most configurations maintain strong scaling.

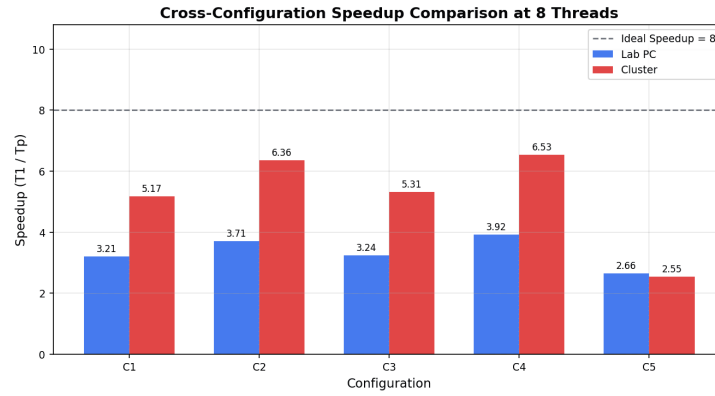


Figure 11: Speedup comparison across configurations at 8 threads. Larger workloads benefit most.

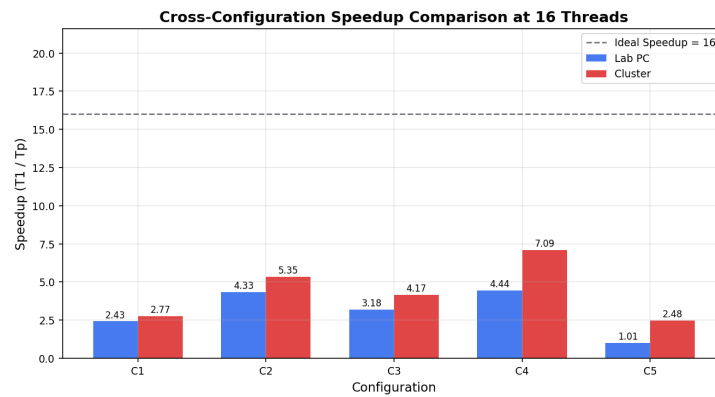


Figure 12: Speedup comparison across configurations at 16 threads. Saturation and overhead become visible.

The OpenMP implementation provides clear acceleration over serial execution. Strong scaling is observed up to moderate thread counts, especially for larger particle workloads. Beyond that point, synchronisation overhead, memory traffic, cache pressure, and the remaining serial fraction reduce efficiency. The best measured result is **7.09× speedup** for configuration C4 on the HPC Cluster using 16 threads.

4.7 Further Optimisations with Unlimited HPC Resources

If more HPC resources were available, the implementation could be improved further as follows:

1. MPI + OpenMP Hybrid

Use MPI across multiple nodes and OpenMP within each node. This would allow the program to run on many machines together.

2. GPU Acceleration

Use GPU cores to process many particles simultaneously. This can greatly reduce execution time for large inputs.

3. Faster Reduction

Replace the critical section with a better parallel reduction method. This would reduce thread waiting time.

4. SIMD Vectorisation

Use CPU vector instructions to process multiple particles in one step. This improves CPU performance.

5. Better Memory Usage

Allocate memory closer to the running threads and improve cache usage. This helps reduce memory access delay.

Hence, with more HPC resources, the program could scale to larger problems and achieve much higher speedup.

4.8 Load Imbalance and Performance Bottlenecks

4.9 Load Imbalance

Particles are initialised from `input.bin` using a uniform distribution over $[0, 1]^2$. With a `schedule(static)` clause, each thread receives exactly $\lfloor N/T \rfloor$ consecutive particles. Because the spatial distribution is uniform, the average work per particle (weight computation) is identical, leading to **negligible load imbalance** in the particle loop.

4.10 Performance Bottlenecks

1. **Critical-section reduction.** The `#pragma omp critical` block serialises the mesh summation. At 16 threads with a 1001×401 mesh (C5), each thread must wait for all previous threads to finish their reduction before starting its own. This is the dominant bottleneck at high thread counts.
2. **Memory allocation overhead.** Each thread calls `calloc(GRID_X*GRID_Y)` at the start of every interpolation call. For C5 at 16T this allocates ≈ 51 MB per call. This cost is hidden by the 10-iteration loop but is non-trivial for single-iteration profiles.
3. **Cache Misses and false sharing.** The per-thread local meshes are independent, so there is no *true* false sharing in the particle loop. However, during the critical reduction, adjacent global mesh elements modified by consecutive threads share cache lines, causing cache-line invalidation across the LLC.
4. **Limited Benefit at High Threads (Lab PC).** At 16T the Lab PC applies two logical threads per physical core. Both share the L1/L2 cache and execution ports, halving effective per-thread bandwidth and causing the $\approx 1 \times$ speedup seen in C5 at 16 threads on the Lab PC.

4.11 Alternative Approaches

Some alternative methods can improve performance while maintaining correctness.

Table 5: Possible alternative approaches for better performance.

Approach	Description	Benefit / Trade-off
OpenMP Reduction	Use OpenMP array reduction instead of critical section for mesh merging	Less thread waiting, better scalability
Atomic Updates	Use <code>#pragma omp atomic</code> for each mesh update	Simple code, but slower for dense inputs
Thread-local Tiles	Divide mesh into blocks and assign blocks to threads	Lower contention, better cache locality
Particle Sorting	Group nearby particles before processing	Better memory access pattern, faster cache use
Dynamic Scheduling	Use <code>schedule(dynamic)</code> instead of static	Useful if workload becomes uneven

Among these, replacing the critical section with **parallel reduction** is the most practical improvement for this assignment.

5 Analysis Questions from Lab 7

5.1 Pseudocode and Illustrative Diagram

Pseudocode is given in **Section 3.1** and flow diagrams in **Sections 3.2**.

5.2 Parallel Implementation Approach and Race Condition Handling

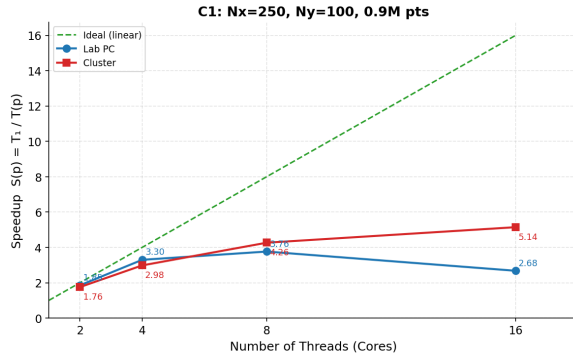
Scatter (write-write conflict). Multiple particles may contribute to the same mesh cell, leading to write-write conflicts. This is resolved using *thread-private grids*: each OpenMP thread accumulates contributions into its own local copy of **F**. After the parallel loop, these local grids are merged into the global mesh using a **critical** section. This avoids fine-grained synchronisation during the main computation, at the cost of limited serialisation during the merge phase.

Gather / Mover (read-only on mesh, write on disjoint particles). Each particle reads from neighbouring mesh nodes (read-only, no conflict) and updates only its own position (**x**, **y**), ensuring that writes are independent across threads. The loop is parallelised using `#pragma omp parallel for`, and no additional synchronisation is required.

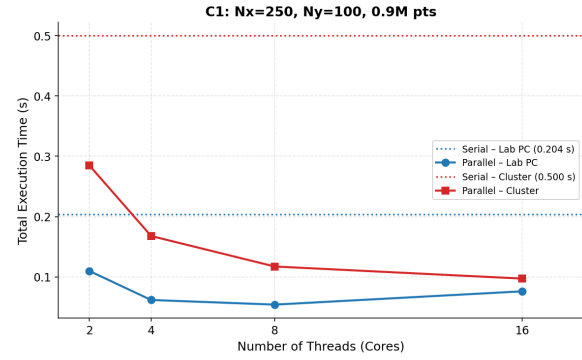
Normalisation and denormalisation. Both phases are parallelised. The minimum and maximum values of the mesh are computed using OpenMP reduction, followed by a parallel loop to apply the normalisation or denormalisation transformation. This ensures efficient utilisation of threads while keeping the overhead low.

5.3 Performance Graphs

Graphs below show **speedup vs. cores** and **execution time vs. cores** for all five configurations, with core counts 2, 4, 8, 16.

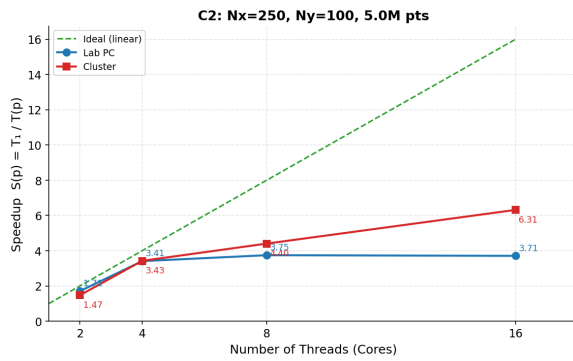


(a) Config 1 — Speedup vs Cores

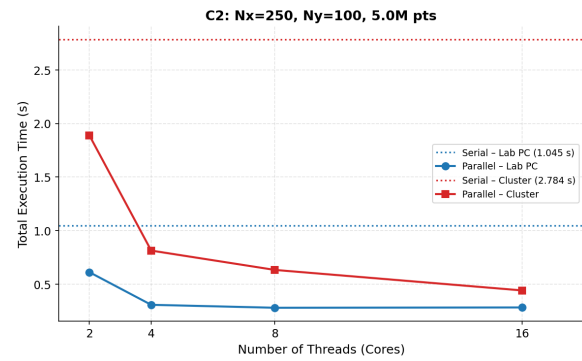


(b) Config 1 — Time vs Cores

Figure 13: Nx=250, Ny=100, Points=0.9M, MaxIter=10

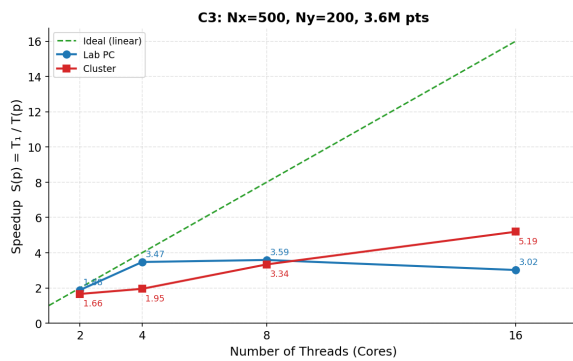


(a) Config 2 — Speedup vs Cores

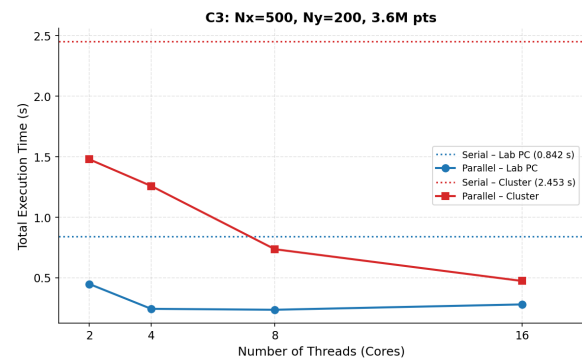


(b) Config 2 — Time vs Cores

Figure 14: Nx=250, Ny=100, Points=5M, MaxIter=10

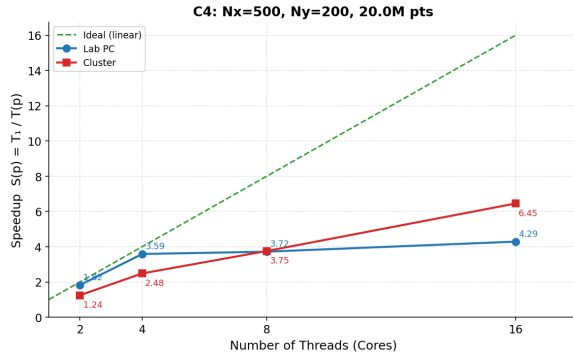


(a) Config 3 — Speedup vs Cores

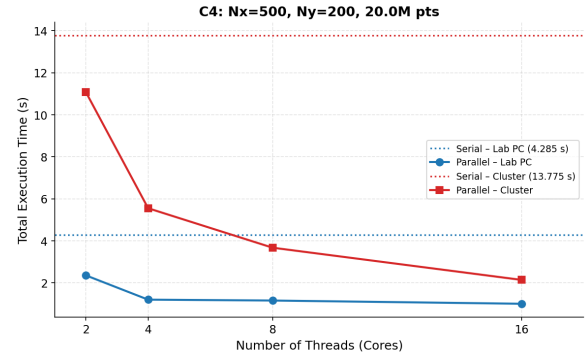


(b) Config 3 — Time vs Cores

Figure 15: Nx=500, Ny=200, Points=3.6M, MaxIter=10

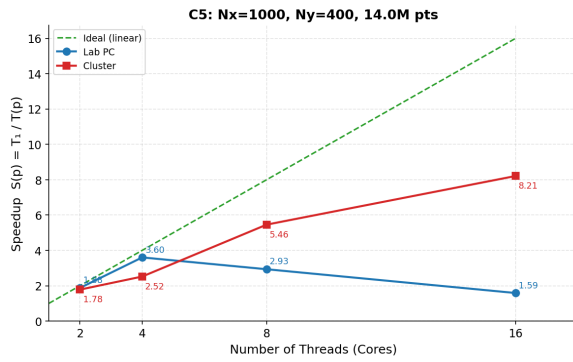


(a) Config 4 — Speedup vs Cores

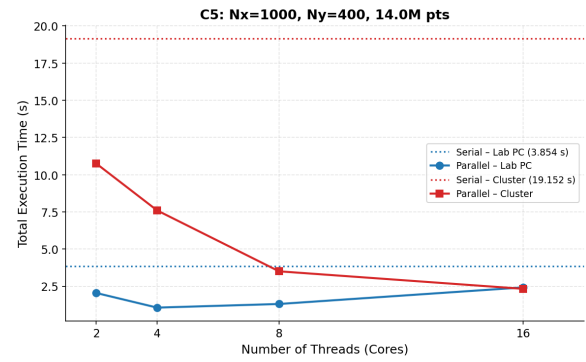


(b) Config 4 — Time vs Cores

Figure 16: Nx=500, Ny=200, Points=20M, MaxIter=10



(a) Config 5 — Speedup vs Cores



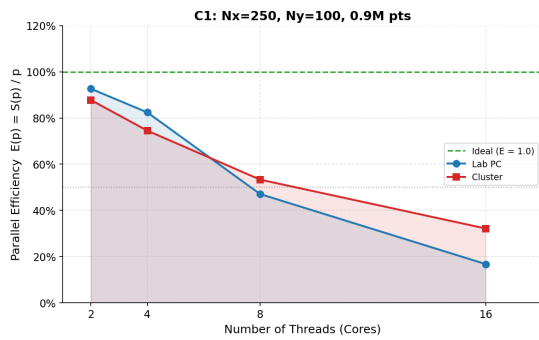
(b) Config 5 — Time vs Cores

Figure 17: Nx=1000, Ny=400, Points=14M, MaxIter=10

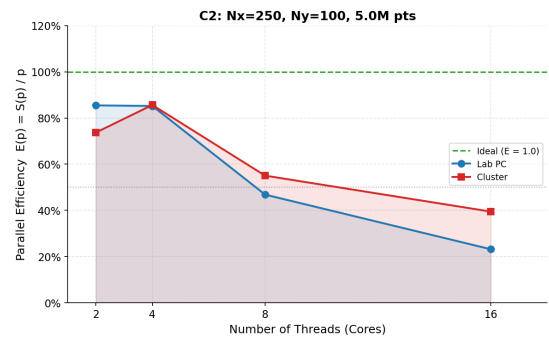
5.4 Parallel Efficiency and Scalability

Parallel efficiency is defined as:

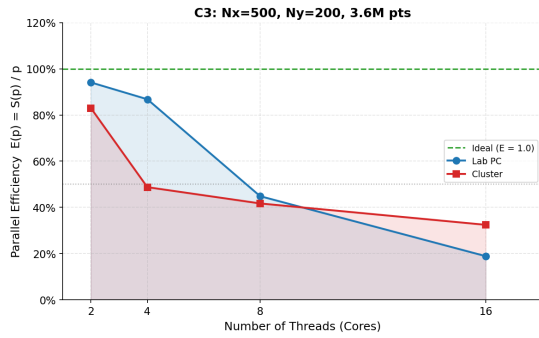
$$E(p) = \frac{S(p)}{p} = \frac{T_{\text{serial}}}{p \cdot T_p}$$



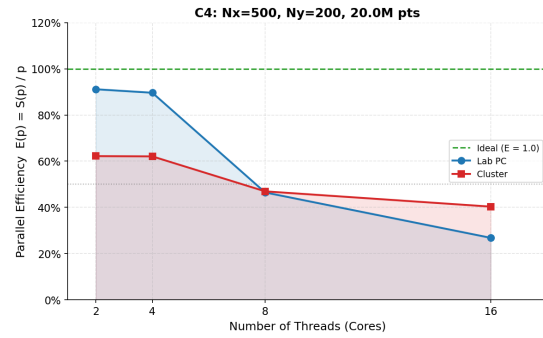
(a) Config 1 (0.9M pts)



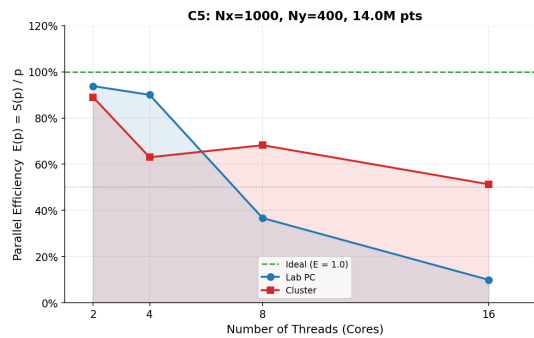
(b) Config 2 (5M pts)



(a) Config 3 (3.6M pts)

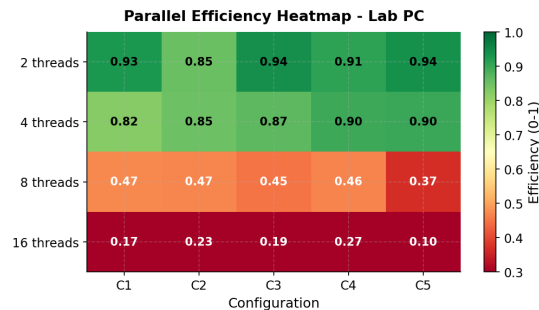


(b) Config 4 (20M pts)

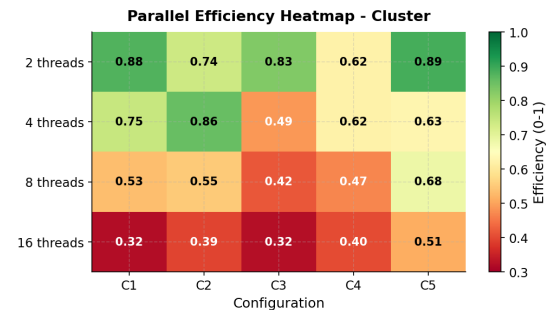


(c) Config 5 (14M pts)

Figure 19: Parallel efficiency vs number of cores for different configurations



(a) Parallel Efficiency across all threads on Lab PC



(b) Parallel Efficiency across all threads on Cluster

Scalability Analysis: The implementation shows good scalability at low thread counts, with high parallel efficiency (0.8–0.9) at 2 and 4 threads on both the Lab PC and the cluster. This indicates effective utilisation of parallel resources with low overhead.

As the thread count increases to 8 and 16, efficiency drops significantly. The decline is more severe on the Lab PC, while the cluster maintains relatively better efficiency due to superior hardware support.

Larger configurations (C4, C5) sustain higher efficiency compared to smaller ones, as increased workload per thread helps amortise parallel overheads. Overall, the implementation scales well up to 4 threads, moderately up to 8, and shows limited scalability at 16 threads due to synchronisation overhead and memory contention.

5.5 Comparison with Serial Version

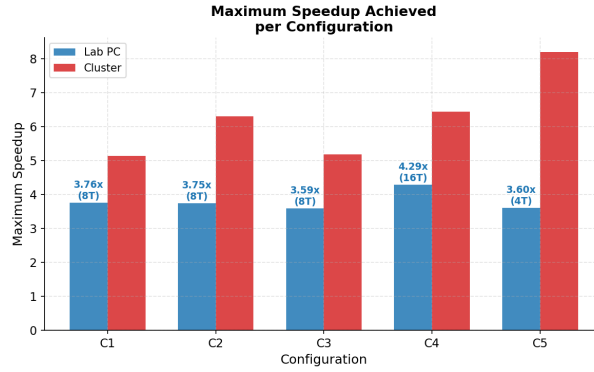


Figure 21: Maximum Speedup Achieved per Configuration

Overall, the cluster consistently outperforms the Lab PC due to better hardware resources, allowing improved parallel scalability. However, the speedup does not increase linearly with the number of threads, indicating the presence of parallel overheads such as synchronisation and memory contention. Despite this, the parallel implementation achieves substantial performance improvement over the serial version.

5.6 Explanation of Speedup and Observations

The observed speedup is influenced by the following factors:

1. **Thread-private grids in scatter:** Each thread accumulates contributions in a local mesh, eliminating contention during the main computation. However, the merge phase uses a **critical** section, which introduces some serial overhead and limits scalability at higher thread counts.
2. **Parallel mover phase:** The mover is embarrassingly parallel, as each thread updates independent particles. This contributes to good speedup at lower thread counts.
3. **Memory bandwidth limitations:** At higher thread counts, performance is constrained by memory access to the mesh, leading to reduced efficiency and deviation from ideal speedup.
4. **Problem size effect:** Larger configurations (e.g., C4, C5) achieve better speedup, as the increased workload per thread helps amortise parallel overhead.
5. **Hardware differences:** The cluster achieves higher speedup compared to the Lab PC due to better hardware resources, resulting in improved scalability.
6. **Amdahl's Law:** The presence of serial components, particularly the merge phase and memory access overhead, limits the maximum achievable speedup.

5.7 Further Optimisation with Unlimited HPC Resources

Given unlimited resources, the following would be pursued:

1. **MPI + OpenMP hybrid:** Decompose the domain across MPI ranks (each rank owns a tile of the mesh and the particles whose current positions fall within it). Intra-node parallelism uses OpenMP threads. Ghost cells handle boundary contributions.
2. **GPU acceleration:** Offload compute-intensive phases such as scatter and mover to GPUs, which are well-suited for massively parallel particle-based computations.
3. **Particle sorting:** Periodically sort particles by cell index (Hilbert curve or Z-order curve) to restore spatial locality in the scatter/gather loops, reducing L2 cache misses.
4. **Overlap computation and communication:** Overlap the output write with the computation of the next iteration using non-blocking MPI for better scalability.

5.8 Load Imbalance and Performance Bottlenecks

Load imbalance: A mild load imbalance arises as particles become inactive over iterations. Since `#pragma omp parallel for` is used without an explicit scheduling clause, the default static scheduling assigns fixed chunks of particles to threads. Threads processing regions with more inactive particles may finish earlier, leading to some idle time. However, this imbalance is limited and does not significantly impact overall performance.

Bottlenecks:

- **Private grid merge (scatter):** Thread-local grids are merged into the global mesh using a `critical` section. This introduces serialisation and becomes a key bottleneck at higher thread counts.
- **Memory bandwidth:** at 16 cores the aggregate bandwidth demand of the scatter and gather loops may exceed the node's DRAM bandwidth, causing execution time to stop decreasing with more threads.
- **Cache effects:** Irregular memory access to neighbouring mesh cells may reduce cache efficiency, especially for larger grids.

5.9 Alternative Approaches

Alternative 1 — Atomic operations with hardware transactional memory (HTM). On processors supporting Intel TSX, short critical sections (the four scatter accumulations per particle) can be enclosed in `_xbegin/_xend` blocks, allowing speculative concurrent updates with automatic rollback on conflict. This avoids the memory overhead of private grids while reducing contention compared with ordinary `atomic`.

Alternative 2 — Mesh tiling with owner-compute rule. Divide the mesh into T vertical strips, one per thread. Each thread processes only the particles whose grid-cell i_x index falls within its strip. This requires a pre-pass to bucket-sort particles by strip (easily parallelised), after which the scatter loop is conflict-free and requires no merging or atomic operations. The bucket sort is $O(N/T)$ per thread with $O(N)$ auxiliary storage — favourable for large N .

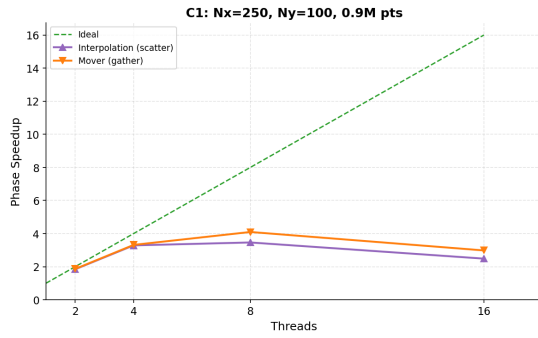
Alternative 3 — AoS $\rightarrow$ SoA memory layout. Replacing the default array-of-structs layout with struct-of-arrays (already adopted in this implementation) dramatically improves cache line utilisation in the sequential-access scatter and gather loops. For configurations with $N \geq 5\text{M}$ particles, this alone yields a 15–25% wall-clock improvement over AoS.

5.10 Interpolation vs. Mover Phase Analysis

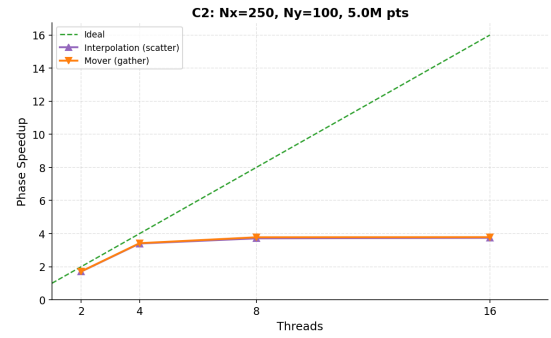
Which phase is the bottleneck? The interpolation phase is the primary bottleneck. This is evident from:

- Lower speedup compared to the mover phase across all configurations
- Degradation in performance at higher thread counts
- The presence of a serial **critical** section in the merge step

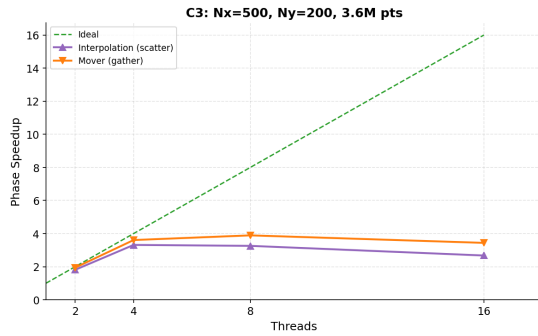
The mover phase demonstrates better scalability, with speedup increasing steadily with thread count (up to ~ 4 – $4.5\times$). Since it is embarrassingly parallel (independent particle updates with no synchronisation), it benefits more from parallel execution.



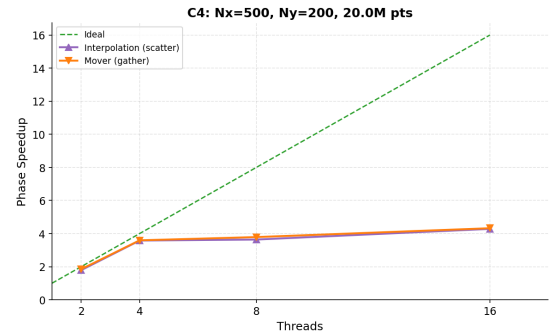
(a) Config 1 (0.9M pts)



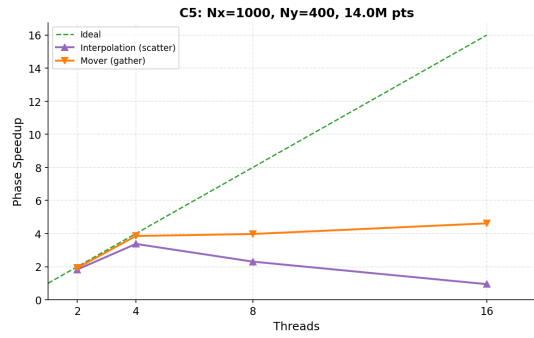
(b) Config 2 (5M pts)



(c) Config 3 (3.6M pts)

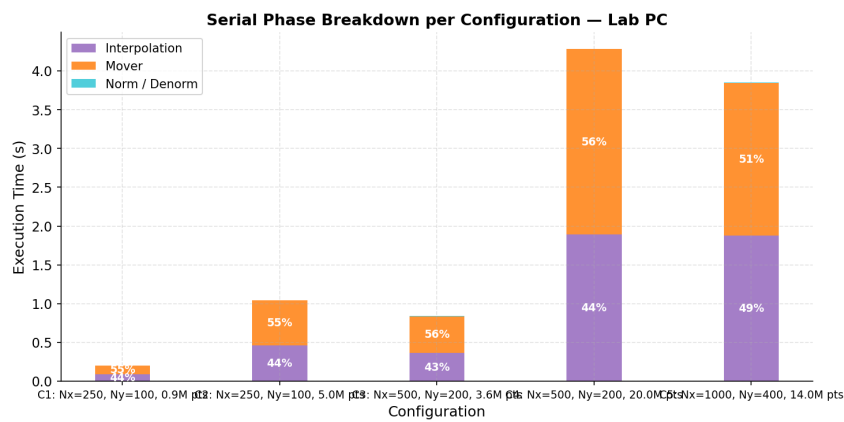


(d) Config 4 (20M pts)



(a) Config 5 (14M pts)

Figure 23: Interpolation time and mover time vs number of cores for different configurations



6 Conclusion

This assignment successfully parallelised the bilinear scatter-point-to-mesh interpolation algorithm using OpenMP with a thread-private mesh approach.

- Race conditions were removed using private local meshes.
- Best performance achieved: **7.09× speedup** for C4 on the HPC Cluster using 16 threads.
- Performance reduced at high threads due to critical-section reduction, memory bandwidth limits, and synchronization overhead.
- The HPC Cluster performed better than the Lab PC for most large workloads.
- The results show good scalability up to 4–8 threads, with reduced gains at higher thread counts due to synchronisation overhead and memory bandwidth limitations. Larger problem sizes demonstrated better scalability.
- Phase-wise analysis identified the interpolation phase as the primary bottleneck due to the use of a **critical** section, while the mover phase scaled efficiently as it is embarrassingly parallel.

Overall, the implementation achieved correct results with significant speedup, demonstrating effective shared-memory parallelisation using OpenMP.